GEEK ON A LEASH

A source-level Rust talk about physical authority

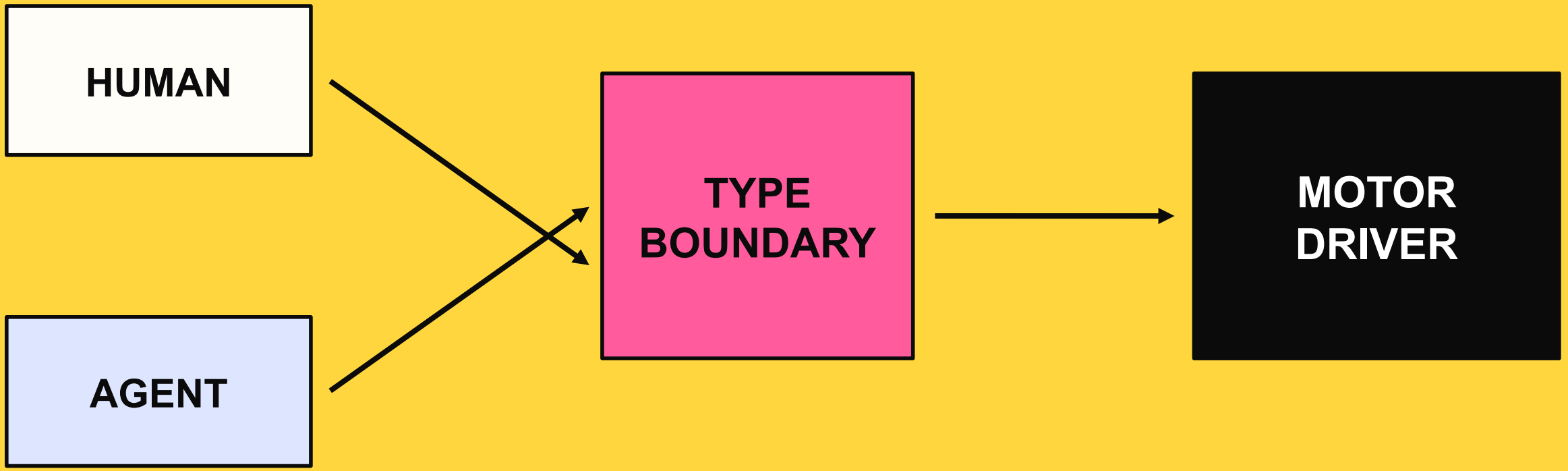
RUST TUESDAYS • 60 MIN

ERIC MANGANARO

Leash + Pinkie

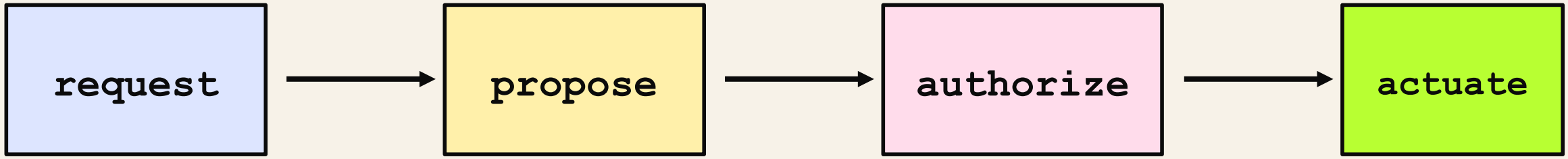
github.com/specdog/leash

Who is allowed to write the motors?



The dangerous bug is ambiguous authority.

The canonical rule is intentionally boring.



No adapter, planner, or GPU kernel may skip a verb.

The types terminate in real hardware.

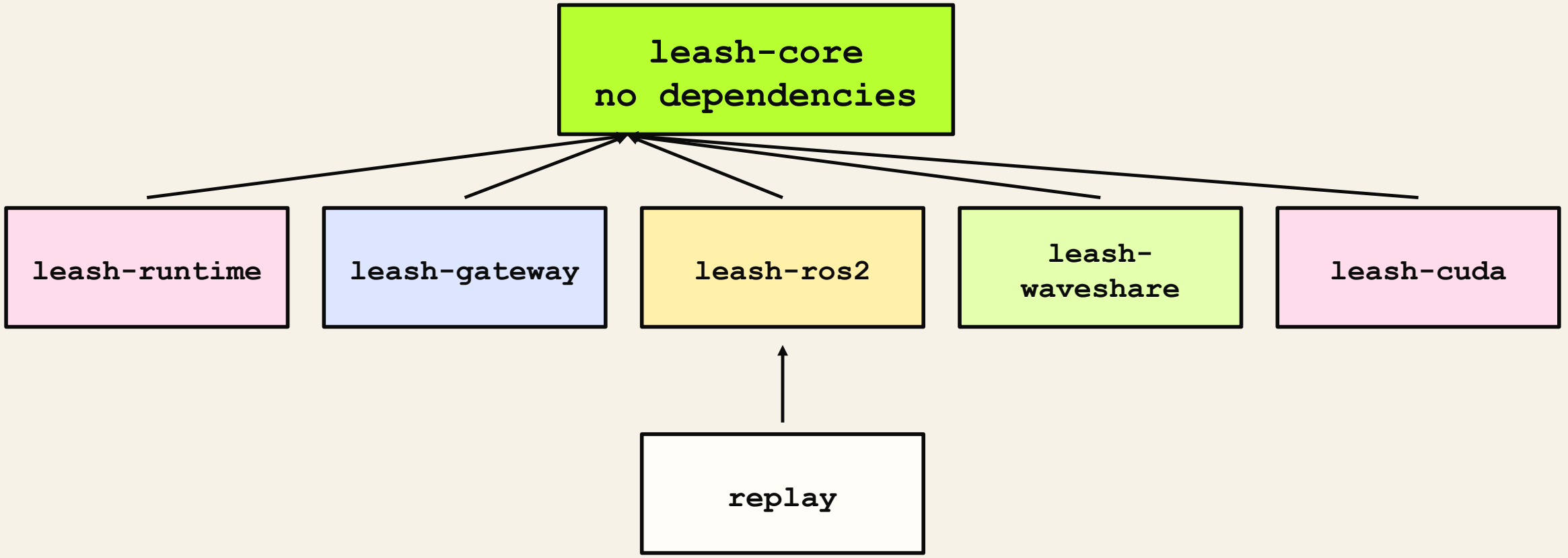
Jetson Orin NX

`serial motor MCU`

LiDAR + camera

A bad abstraction becomes a moving machine.

The crate graph is an authority graph.



Dependencies point inward. Physical authority stays narrow.

Use the compiler to make illegal motion awkward.

`NEWTYPE`	validate once; keep raw floats out
`TYPESTATE`	authorization cannot be forged
`TRAITS`	hardware varies; contract does not
`OWNERSHIP`	one thread owns physical I/O
`DROP`	shutdown is part of the type lifecycle

Unsafe is denied by default, then isolated.

crate roots

```
// leash-runtime/src/lib.rs
#![forbid(unsafe_code)]

// leash-waveshare/src/lib.rs
#![forbid(unsafe_code)]

// leash-cuda/src/lib.rs
#![deny(unsafe_code)]

#[cfg(feature = "cuda")]
#[allow(unsafe_code)]
mod device;
```

The default is a compiler error.

The exception is named and feature-gated.

Review the island, not the ocean.

A private field turns validation into a constructor.

leash-core/src/drive.rs

```
#[derive(Debug, Clone, Copy, Default, PartialEq, PartialOrd)]
pub struct NormalizedDrive(f64);

impl NormalizedDrive {
    pub const ZERO: Self = Self(0.0);

    pub fn new(value: f64) -> Result<Self, DomainError> {
        if !value.is_finite() {
            return Err(DomainError::NonFinite("normalized
drive"));
        }
        if !(-1.0..=1.0).contains(&value) {
            return Err(DomainError::OutOfRange("normalized
drive"));
        }
        Ok(Self(value))
    }
}
```

Tuple field is private.

All construction crosses one gate.

Downstream code receives a proof.

Derives are an API decision, not decoration.

leash-core/src/drive.rs

```rust
#[derive(Debug, Clone, Copy, Default, PartialEq,
PartialOrd)]
pub struct NormalizedDrive(f64);

#[derive(Debug, Clone, Copy, Default, PartialEq)]
pub struct DifferentialDrive {
    pub left: NormalizedDrive,
    pub right: NormalizedDrive,
}

impl DifferentialDrive {
    pub const STOP: Self = Self {
        left: NormalizedDrive::ZERO,
        right: NormalizedDrive::ZERO,
    };
}
```

Copy is safe because the value is tiny and immutable.

PartialEq supports exact protocol tests.

STOP is a canonical, allocation-free value.

One macro stamps the same invariant onto every unit.

leash-core/src/units.rs

```
macro_rules! finite_unit {
    ($name:ident, $label:literal) => {
        #[derive(Clone, Copy, Debug, PartialEq)]
        pub struct $name(f64);

        impl $name {
            pub fn new(value: f64) -> Result<Self,
DomainError> {
                if !value.is_finite() {
                    return
Err(DomainError::NonFinite($label));
                }
                Ok(Self(value))
            }
        }
    };
}
```

macro_rules! removes invariant drift.

Each expansion is still a distinct type.

Meters cannot silently become radians.

NonZeroU64 removes an invalid state from memory.

leash-core/src/time.rs

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd,
Ord, Hash)]
pub struct Sequence(NonZeroU64);

impl Sequence {
    pub fn new(value: u64) -> Result<Self, DomainError> {
        NonZeroU64::new(value)
            .map(Self)
            .ok_or(DomainError::Zero("sequence"))
    }

    pub fn next(self) -> Result<Self, DomainError> {
        self.get()
            .checked_add(1)
            .ok_or(DomainError::Overflow("sequence"))
            .and_then(Self::new)
    }
}
```

Zero cannot be represented.

checked_add makes overflow explicit.

Option says absence is ordinary, not exceptional.

Time arithmetic is checked and domain-specific.

leash-core/src/time.rs

```
#[derive(Debug, Clone, Copy, Default, PartialEq, Eq, PartialOrd,
Ord, Hash)]
pub struct MonotonicNanos(u64);

impl MonotonicNanos {
    pub fn checked_add(self, duration: DurationNanos)
        -> Result<Self, DomainError> {
        self.0
            .checked_add(duration.0)
            .map(Self)
            .ok_or(DomainError::Overflow("monotonic timestamp"))
    }

    pub fn duration_since(self, earlier: Self)
        -> Result<DurationNanos, DomainError> {
        self.0
            .checked_sub(earlier.0)
            .map(DurationNanos)
            .ok_or(DomainError::TimeReversed)
    }
}
```

Monotonic is not wall-clock time.

Subtraction cannot go negative unnoticed.

Overflow is visible in the return type.

Stamped<T>::map moves metadata without cloning it.

leash-core/src/time.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Stamped<T> {
    pub at: MonotonicNanos,
    pub sequence: Sequence,
    pub value: T,
}

impl<T> Stamped<T> {
    pub fn map<U>(self, transform: impl FnOnce(T) -> U) ->
Stamped<U> {
        Stamped {
            at: self.at,
            sequence: self.sequence,
            value: transform(self.value),
        }
    }
}
```

self is consumed: no partial alias survives.

FnOnce permits transforms that consume captures.

Only the payload type changes.

Candidate<C> carries intent, identity, and a deadline.

leash-core/src/drive.rs

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct Candidate<C> {
    pub id: CommandId,
    pub issued_at: MonotonicNanos,
    pub deadline: MonotonicNanos,
    pub command: C,
}

impl<C> Candidate<C> {
    pub const fn new(
        id: CommandId,
        issued_at: MonotonicNanos,
        deadline: MonotonicNanos,
        command: C,
    ) -> Self {
        Self { id, issued_at, deadline, command }
    }
}
```

C keeps the envelope command-agnostic.

CommandId binds producer epoch and sequence.

A proposal is data, never authority.

Authorization is a value you cannot construct outside core.

leash-core/src/drive.rs

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct Authorized<C> {
    command_id: CommandId,
    evidence_id: EvidenceId,
    authorized_at: MonotonicNanos,
    command: C,
}

impl<C> Authorized<C> {
    pub const fn command_id(&self) -> CommandId {
        self.command_id
    }

    pub const fn evidence_id(&self) -> EvidenceId {
        self.evidence_id
    }

    pub fn command(&self) -> &C {
        &self.command
    }
}
```

Private fields block struct literals downstream.

Accessors expose facts, not construction power.

The type is a capability token.

The gate consumes a proposal and returns a capability.

leash-core/src/drive.rs

```
pub fn authorize<C>(
    &mut self,
    candidate: Candidate<C>,
    now: MonotonicNanos,
) -> Result<Authorized<C>, SafetyDenial> {
    match self.state {
        SafetyState::Disarmed => return Err(SafetyDenial::Disarmed),
        SafetyState::EStopped => return Err(SafetyDenial::EStopped),
        SafetyState::Faulted => return Err(SafetyDenial::Faulted),
        SafetyState::Ready | SafetyState::Moving => {}
    }

    if candidate.issued_at > now {
        return Err(SafetyDenial::IssuedInFuture);
    }
    if candidate.deadline < now {
        return Err(SafetyDenial::Expired);
    }

    let evidence_id = self.next_evidence_id()?;
    Ok(Authorized {
        command_id: candidate.id,
        evidence_id,
        authorized_at: now,
        command: candidate.command,
    })
}
```

candidate is moved: it cannot be reused accidentally.

match is exhaustive over safety state.

? exits on the first failed proof.

The public API is tested by code that must not compile.

leash-core/src/drive.rs doctest

```
/// \`\`\`compile_fail
/// use leash_core::DifferentialDrive;
/// let _command = DifferentialDrive::new(0.5, 0.5);
/// \`\`\`

/// \`\`\`compile_fail
/// use leash_core::{Authorized, DifferentialDrive};
/// let _forged = Authorized::<DifferentialDrive> {
///     command: DifferentialDrive::STOP,
/// };
/// \`\`\`
```

Documentation becomes a negative contract.

Refactors fail if the forbidden path opens.

The compiler is part of the test harness.

Frame<Tag> stores no tag at runtime—and still separates frames.

leash-core/src/frame.rs

```
pub enum Map {}
pub enum Odom {}
pub enum Base {}
pub enum Sensor {}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Frame<Tag> {
    name: FrameName,
    marker: PhantomData<fn() -> Tag>,
}

#[derive(Debug, Clone, PartialEq)]
pub struct Pose2<Tag> {
    pub frame: Frame<Tag>,
    pub x: Meters,
    pub y: Meters,
    pub yaw: Radians,
}
```

Empty enums are type-level labels.

PhantomData ties Tag to ownership/type rules.

No tag bytes are stored in Pose2.

A pose in Odom is not a pose in Map.

leash-core/src/frame.rs doctest

```
/// \`\`\`compile_fail
/// use leash_core::{Frame, FrameName, Map, Meters, Odom,
Pose2, Radians};
/// fn consume_map(_: Pose2<Map>) {}
/// let odom =
Frame::<Odom>::new(FrameName::new("odom").unwrap());
/// let pose = Pose2::new(
///     odom,
///     Meters::new(0.0).unwrap(),
///     Meters::new(0.0).unwrap(),
///     Radians::new(0.0).unwrap(),
/// );
/// consume_map(pose);
/// \`\`\`
```

Same fields do not mean same semantics.

The missing transform becomes a compiler error.

Zero runtime checks; strong compile-time separation.

Supertraits define the minimum hardware capability.

leash-waveshare/src/lib.rs

```
pub trait ControllerIo: Read + Write + Send {}

impl<T> ControllerIo for T
where
    T: Read + Write + Send,
{}

pub trait ControllerIoFactory: Send {
    fn open(&mut self) -> io::Result<Box<dyn ControllerIo>>;
}
```

Read + Write + Send is the complete capability set.

The blanket impl admits every compatible transport.

Box<dyn Trait> erases the concrete serial type.

A closure can be the reconnection factory.

leash-waveshare/src/lib.rs

```
pub trait ControllerIoFactory: Send {
    fn open(&mut self) -> io::Result<Box<dyn ControllerIo>>;
}

impl<F> ControllerIoFactory for F
where
    F: FnMut() -> io::Result<Box<dyn ControllerIo>> + Send,
{
    fn open(&mut self) -> io::Result<Box<dyn ControllerIo>> {
        self()
    }
}
```

FnMut permits reconnect state to evolve.

Send permits ownership transfer to the I/O thread.

The trait object keeps Supervisor non-generic at runtime.

The actuation port chooses its acknowledgement protocol.

leash-runtime/src/supervisor.rs

```
pub trait ActuationAcknowledgement: Send + 'static {
    fn applied(&self) -> bool;
    fn verified_zero(&self) -> bool;

    fn command_id(&self) -> Option<CommandId> { None }
    fn evidence_id(&self) -> Option<EvidenceId> { None }
    fn applied_sequence(&self) -> Option<u64> { None }
}

pub trait ActuationPort: Send + 'static {
    type Acknowledgement: ActuationAcknowledgement;
    type Error: fmt::Display + Send + 'static;

    fn submit_drive(
        &mut self,
        command: Authorized<DifferentialDrive>,
    ) -> Result<(), Self::Error>;

    fn try_acknowledgement(
        &mut self,
    ) -> Result<Option<Self::Acknowledgement>, Self::Error>;
}
```

Associated types bind one Ack and Error to each port.

The port accepts only Authorized<Drive>.

'static permits the whole port to live on a thread.

An equality bound connects the supervisor to the port.

leash-runtime/src/supervisor.rs

```
pub struct CpuSafetySupervisor<A>
{
    handle: SupervisorHandle,
    events: Option<LatestReader<SupervisorEvent<A>>>,
    worker: Option<JoinHandle<()>>,
}

impl<A> CpuSafetySupervisor<A>
where
    A: ActuationAcknowledgement,
{
    pub fn spawn<P>(
        kernel: ControlKernel,
        port: P,
        clock: Box<dyn Clock + Send>,
        config: SupervisorConfig,
    ) -> Result<Self, SupervisorStartError>
    where
        P: ActuationPort<Acknowledgement = A>,
    {
        Self::spawn_inner(kernel, port, clock, config, None)
    }
}
```

A is named once on the supervisor state.

P stays generic only at construction.

Acknowledgement = A is an associated-type equality.

Static dispatch inside; dynamic dispatch at hardware seams.

MONOMORPHIZED

```
fn run_supervisor<P>(
    mut port: P,
    // ...
) where
    P: ActuationPort,
{
    // specialized for P
}
```

ERASED

```
fn controller_loop(
    io: Box<dyn ControllerIo>,
) {
    // one runtime-selected I/O
object
}
```

FAST CONTROL PATH

REPLACEABLE ADAPTER

Choose dispatch based on ownership and change boundaries—not dogma.

move transfers the physical port into one owner thread.

leash-runtime/src/supervisor.rs [abridged]

```
let panic_shared = Arc::clone(&shared);
let thread_shared = Arc::clone(&shared);

let worker = thread::Builder::new()
    .name("leash-cpu-safety".to_string())
    .spawn(move || {
        let _ = thread_shared.worker_thread.set(thread::current());
        let result = std::panic::catch_unwind(
            std::panic::AssertUnwindSafe(|| {
                run_supervisor(
                    kernel, port, clock, config,
                    proposal_receiver, safety_receiver,
                    event_publisher, evidence, shared,
                );
            }),
        );
        if result.is_err() {
            panic_shared.faulted.store(true, Ordering::Release);
            panic_shared.closed.store(true, Ordering::Release);
        }
    })
    .map_err(|error| SupervisorStartError::Thread(error.to_string()))?;
```

move gives the thread exclusive port ownership.

Arc shares state, not mutable hardware access.

catch_unwind converts panic into a fail-closed outcome.

Drop makes thread shutdown part of ownership.

leash-runtime/src/supervisor.rs

```
impl<A> Drop for CpuSafetySupervisor<A>
{
    fn drop(&mut self) {
        self.handle.shared.shutdown.store(true,
Ordering::Release);
        self.handle.shared.wake();

        if let Some(worker) = self.worker.take() {
            let _ = worker.join();
        }
    }
}
```

Dropping sets the shutdown flag.

Wake prevents a sleeping loop from hanging.

Option::take guarantees one join attempt.

Backpressure errors return ownership to the sender.

leash-runtime/src/lane.rs

```
pub enum OverflowPolicy {
    RejectNewest,
    DropOldest,
}

pub enum SendOutcome<T> {
    Enqueued,
    ReplacedOldest(T),
}

pub enum SendError<T> {
    Closed(T),
    Full(T),
}

struct LaneState<T> {
    queue: VecDeque<T>,
    high_watermark: usize,
    sent: u64,
    received: u64,
    rejected: u64,
    dropped: u64,
}

struct LaneShared<T> {
    capacity: usize,
    policy: OverflowPolicy,
    closed: AtomicBool,
    state: Mutex<LaneState<T>>,
}
```

The queue has a hard capacity.

Errors preserve the rejected T.

Replacement reports the evicted T.

Overflow policy is an exhaustive match, not a comment.

leash-runtime/src/lane.rs [abridged]

```
pub fn try_send(&self, value: T)
    -> Result<SendOutcome<T>, SendError<T>> {
    if self.shared.closed.load(Ordering::Acquire) {
        return Err(SendError::Closed(value));
    }

    let mut state = lock(&self.shared.state);
    if self.shared.closed.load(Ordering::Acquire) {
        return Err(SendError::Closed(value));
    }

    let outcome = if state.queue.len() == self.shared.capacity {
        match self.shared.policy {
        OverflowPolicy::RejectNewest => {
            state.rejected = state.rejected.saturating_add(1);
            return Err(SendError::Full(value));
        }
        OverflowPolicy::DropOldest => {
            let replaced = state.queue.pop_front()
                .expect("a full bounded queue contains an item");
            state.dropped = state.dropped.saturating_add(1);
            state.queue.push_back(value);
            SendOutcome::ReplacedOldest(replaced)
        }
        }
    } else {
        state.queue.push_back(value);
        SendOutcome::Enqueued
    };
    Ok(outcome)
}
```

Check closed before taking the mutex.

The match documents both overload semantics.

No unbounded queue can consume the device.

Safety requests bypass the ordinary proposal lane.

leash-runtime/src/safety.rs

```
struct SafetyShared {
    stop: AtomicU64,
    estop: AtomicU64,
    closed: AtomicBool,
}

impl SafetySender {
    pub fn request(&self, kind: SafetyKind)
        -> Result<u64, SafetyRequestError> {
        if self.shared.closed.load(Ordering::Acquire) {
            return Err(SafetyRequestError::Closed);
        }
        let sequence = match kind {
            SafetyKind::Stop => increment(&self.shared.stop),
            SafetyKind::EStop => increment(&self.shared.estop),
        }?;
        if self.shared.closed.load(Ordering::Acquire) {
            return Err(SafetyRequestError::Closed);
        }
        Ok(sequence)
    }
}

fn increment(counter: &AtomicU64) -> Result<u64, SafetyRequestError> {
    counter.fetch_update(Ordering::AcqRel, Ordering::Acquire,
        |current| current.checked_add(1))
        .map(|previous| previous + 1)
        .map_err(|_| SafetyRequestError::SequenceExhausted)
}
```

Safety has a dedicated mailbox.

AcqRel makes increment a read-modify-write boundary.

fetch_update prevents sequence wraparound.

The receiver checks E-stop before stop.

leash-runtime/src/safety.rs

```
pub fn try_recv(&mut self)
    -> Result<Option<SafetySignal>, SafetyReceiveError> {
    let estop = self.shared.estop.load(Ordering::Acquire);
    if estop > self.seen_estop {
        let signal = signal(SafetyKind::EStop, self.seen_estop, estop);
        self.seen_estop = estop;
        return Ok(Some(signal));
    }

    let stop = self.shared.stop.load(Ordering::Acquire);
    if stop > self.seen_stop {
        let signal = signal(SafetyKind::Stop, self.seen_stop, stop);
        self.seen_stop = stop;
        return Ok(Some(signal));
    }

    if self.shared.closed.load(Ordering::Acquire) {
        return Err(SafetyReceiveError::Closed);
    }
    Ok(None)
}
```

E-stop wins if both flags are present.

Per-kind counters preserve coalesced requests.

Closed is distinct from no pending signal.

Option::replace makes latest-only semantics explicit.

leash-runtime/src/latest.rs

```
pub fn publish(&self, value: Stamped<T>)
    -> Result<Option<Stamped<T>>, PublishError<T>> {
    let mut state = lock(&self.state);

    if state.last_sequence
        .is_some_and(|sequence| value.sequence <= sequence) {
        state.rejected_out_of_order =
            state.rejected_out_of_order.saturating_add(1);
        return Err(PublishError::SequenceNotIncreasing(value));
    }

    state.last_sequence = Some(value.sequence);
    state.published = state.published.saturating_add(1);
    let replaced = state.value.replace(value);
    if replaced.is_some() {
        state.replaced = state.replaced.saturating_add(1);
    }
    Ok(replaced)
}

pub fn take(&mut self) -> Option<Stamped<T>> {
    let mut state = lock(&self.state);
    let value = state.value.take();
    if value.is_some() {
        state.taken = state.taken.saturating_add(1);
    }
    value
}
```

Stale sequence numbers are rejected.

replace returns the displaced observation.

take moves the newest value out exactly once.

Serde turns the network shape into an enum.

leash-gateway/src/lib.rs

```rust
#[derive(Debug, Clone, PartialEq, Deserialize, Serialize)]
#[serde(
    tag = "command",
    rename_all = "snake_case",
    deny_unknown_fields
)]
pub enum CommandRequest {
    Authorize {
        operator: String,
        expires_at_ns: u64,
    },
    Drive {
        left: f64,
        right: f64,
        deadline_ns: u64,
    },
    Stop {},
    EStop {},
    ResetEStop { approved: bool },
    SetPlannerActive { active: bool },
    CancelPlanner {},
}
```

Tagged enum gives one explicit command discriminator.

deny_unknown_fields rejects schema drift.

Decode errors enter the typed error path.

map_err and ? preserve context without nesting.

leash-gateway/src/lib.rs

```
pub enum GatewayError {
    InvalidJson(Box<str>),
    InvalidDomain(Box<str>),
    Proposal(SupervisorSubmitError),
    Safety(Box<str>),
    Timeout,
    Supervisor(Box<str>),
    Encode(Box<str>),
}

pub fn decode_and_execute(&self, json: &[u8])
    -> Result<Vec<u8>, GatewayError> {
    let request = serde_json::from_slice(json)
        .map_err(|error| GatewayError::InvalidJson(
            error.to_string().into_boxed_str()
        ))?;
    let response = self.execute(request)?;
    serde_json::to_vec(&response)
        .map_err(|error| GatewayError::Encode(
            error.to_string().into_boxed_str()
        ))
}
```

The enum retains the failure category.

? keeps the happy path visually flat.

Boundary details become owned Box<str> values.

A missing acknowledgement ticket exits immediately.

leash-waveshare/src/lib.rs

```
fn try_acknowledgement(&mut self)
    -> Result<Option<Self::Acknowledgement>, Self::Error> {
    let Some(ticket) = self.pending.front() else {
        return Ok(None);
    };

    match ticket.try_take().map_err(PortError::Submit)? {
        Some(ack) => {
            self.pending.pop_front();
            Ok(Some(ack))
        }
        None => Ok(None),
    }
}
```

let-else handles the precondition at the top.

Empty is normal; disconnected is an error.

front borrows; pop_front happens only after an acknowledgement.

specdog/leash • 566bc569

The hardware adapter maps concrete protocol into the trait.

leash-waveshare/src/lib.rs

```
impl ActuationAcknowledgement for CommandAck {
    fn applied(&self) -> bool {
        self.outcome == AckOutcome::Applied
    }
    fn verified_zero(&self) -> bool {
        self.verified_zero
    }
    fn command_id(&self) -> Option<CommandId> {
        Some(self.command_id)
    }
}

impl ActuationPort for WaveshareActuationPort {
    type Acknowledgement = CommandAck;
    type Error = PortError;

    fn submit_drive(
        &mut self,
        command: Authorized<DifferentialDrive>,
    ) -> Result<(), Self::Error> {
        let ticket = self.handle.submit_drive(command)
            .map_err(PortError::Submit)?;
        self.pending.push_back(ticket);
        Ok(())
    }
}
```

Associated types become concrete here.

Authorization remains present at the last adapter.

Hardware errors are not erased prematurely.

Navigation proposals carry their frame in the type.

leash-ros2/src/lib.rs

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct PathProposal {
    pub frame: Frame<Map>,
    pub at: MonotonicNanos,
    pub poses: Box<[Pose2<Map>]>,
}

#[derive(Debug, Clone, PartialEq)]
pub struct PlanarTransform<Parent, Child> {
    pub parent: Frame<Parent>,
    pub child: Frame<Child>,
    pub at: MonotonicNanos,
    pub x: Meters,
    pub y: Meters,
    pub yaw: Radians,
}

#[derive(Debug, Clone, PartialEq)]
pub struct NavigationGoal {
    pub activity_id: ActivityId,
    pub pose: Pose2<Map>,
    pub received_at: MonotonicNanos,
}
```

Map is encoded in every navigation pose.

Transforms name both parent and child types.

ROS2 is an adapter—not an authority bypass.

The unsafe launch is narrow, validated, and non-authoritative.

leash-cuda/src/device.rs [abridged]

```
let output_count = cells.len()
    .checked_mul(depth as usize)
    .ok_or(ComputeInputError::LengthOverflow)?;
let cell_count = u32::try_from(cells.len())
    .map_err(|_| WorkError::InvalidInput(
        ComputeInputError::LengthOverflow))?;
let launch_count = u32::try_from(output_count)
    .map_err(|_| WorkError::InvalidInput(
        ComputeInputError::LengthOverflow))?;

ensure_capacity(&self.stream,
    &mut self.occupancy_output, output_count)?;

let cells_view = self.occupancy_cells.slice(..cells.len());
let mut output_view =
    self.occupancy_output.slice_mut(..output_count);

{
    unsafe {
        self.stream.launch_builder(&self.project_occupancy)
            .arg(&cells_view)
            .arg(&mut output_view)
            .arg(&cell_count)
            .arg(&depth)
            .launch(LaunchConfig::for_num_elems(launch_count))
    }
    .map_err(|error| backend_error("launch project_occupancy", error))?;
}

let output_view = self.occupancy_output.slice(..output_count);
self.stream.clone_dtoh(&output_view)
    .map_err(|error| backend_error("download output", error))
}
```

Validation sits immediately before unsafe.

Typed CudaSlice views bound the launch arguments.

CUDA computes shadow evidence; CPU authorizes motion.

Determinism and measurements close the loop.

leash-replay/src/lib.rs

```
#[derive(Debug, Clone, Copy)]
struct StableDigest(u64);

impl StableDigest {
    const OFFSET: u64 = 0xcbf29ce484222325;
    const PRIME: u64 = 0x000001000000001b3;

    fn u64(&mut self, value: u64) {
        for byte in value.to_le_bytes() {
            self.0 ^= u64::from(byte);
            self.0 = self.0.wrapping_mul(Self::PRIME);
        }
    }
}

let first = scenario.verify().unwrap();
let second = scenario.verify().unwrap();
assert_eq!(first, second);
```

58.306 µs
p99 transition

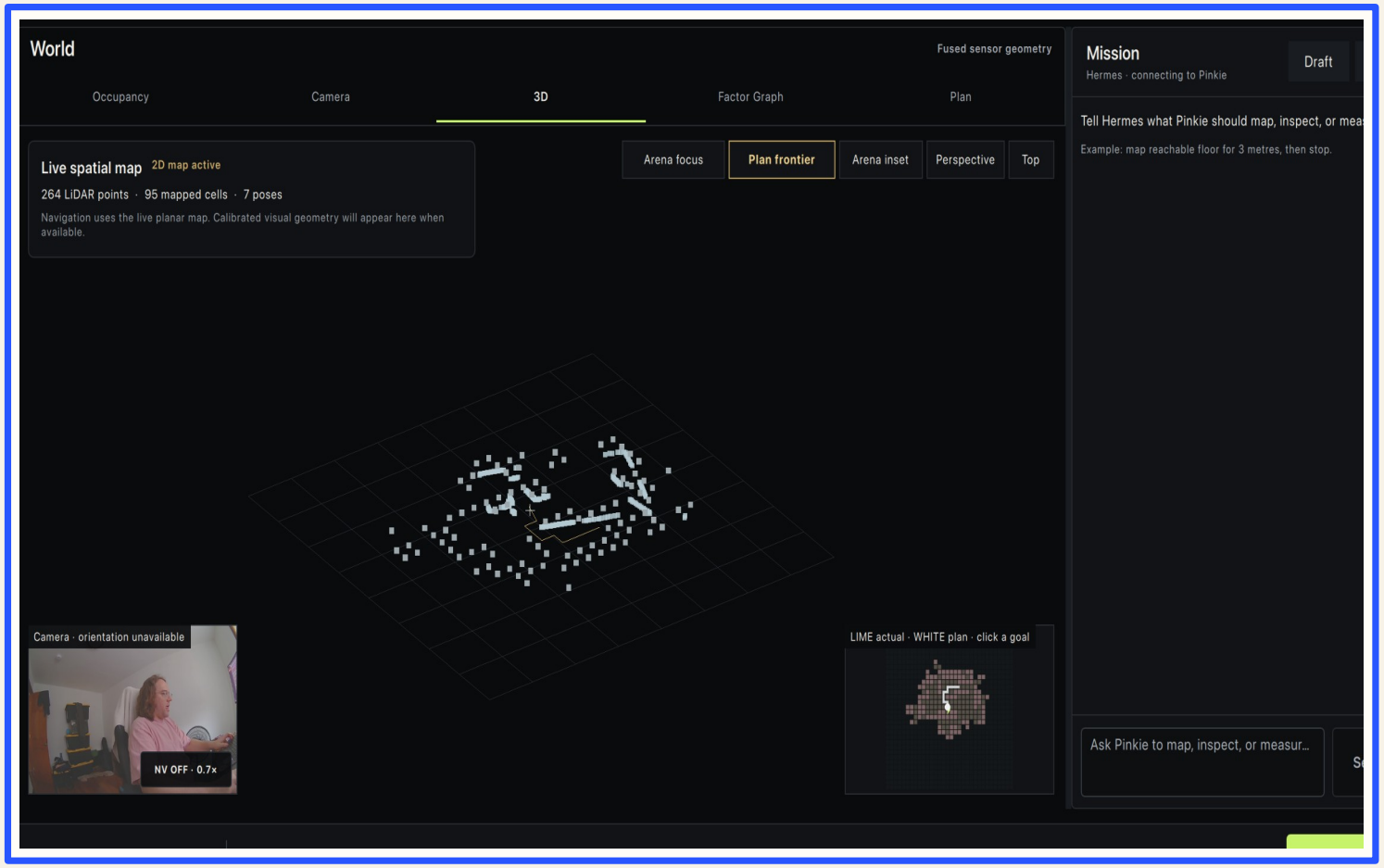
0
deadline misses

110,293
durable records/s

37.622 ms
physical E-stop ack

Same input → same transitions → same digest

Qualia reasons. Leash retains physical authority.



MISSION ONTOLOGY SEMANTICS

↓

PROPOSALS + EVIDENCE

↓

LEASH AUTHORIZES

Demo the boundary, not a heroic autonomy claim.

1. read-only preflight
2. show proposal + evidence
3. approve one ≤ 0.10 pulse
4. verify zero + acknowledgement

operator contract

```
if preflight.is_red() {
    play_recorded_fallback();
    return;
}

assert!(operator_token.is_active());
assert!(pulse.drive <= 0.10);
assert!(pulse.duration <= 500_ms);

let evidence = run_once(pulse)?;
assert_eq!(evidence.final_drive,
STOP);
```

Never improvise movement on stage.

WRITE THE AUTHORITY RULE IN TYPES.

- NEWTYPES VALIDATE
- TYPESTATE AUTHORIZES
- OWNERSHIP ISOLATES
- DROP CLOSES
- EVIDENCE PROVES

SLIDES + CODE

Direct PDF QR • valid through Sep 8, 2026 12:59 PM ET

github.com/specdog/leash